

Web Development

Labo 13

Jari Rhellam

Opdracht 1

Check!!

1. Functies

Functiedefinities

- **Functie declaratie:**

```
function sayHello() {  
    console.log("Hello!");  
}
```

Een functie declareert alleen code; hij voert deze niet uit totdat je hem aanroept:

```
sayHello(); // Roep de functie aan
```

1.2 Parameters en argumenten

- Functies kunnen **inputs** krijgen via **parameters**:

```
function greet(name) {  
    console.log("Hello, " + name);  
}  
greet("Nick"); // Output: Hello, Nick
```

De waarde die je meegeeft bij het aanroepen heet een argument.

1.3 Default parameters

- Als geen argument wordt meegegeven, kan een parameter een **default waarde** krijgen:

```
function greet(name = "stranger") {  
    console.log("Hello, " + name);  
}  
greet(); // Output: Hello, stranger
```

1.4 Return

- Functies kunnen **resultaten teruggeven** met return:

```
function square(num) {  
  return num * num;  
}  
let result = square(5); // result = 25
```

Return stopt de functie; code na return wordt niet uitgevoerd.

1.5 Helper functies

- Functies kunnen andere functies **aanroepen**:

```
function multiplyByTwo(x) {  
  return x * 2;  
}  
function addTen(x) {  
  return multiplyByTwo(x) + 10;  
}  
addTen(5); // 20
```

1.6 Function expressions

- Een **function expression** slaat een anonieme functie op in een variabele:

```
const sayHello = function(name) {  
  console.log("Hello, " + name);  
};
```

Function expressions worden niet gehost, dus je kan ze pas gebruiken na declaratie.

1.7 Arrow functies

- Kortere syntaxis voor functies (ES6):

```
const square = (num) => {  
  return num * num;  
};
```

Concise body: als de functie maar 1 regel is, kan return en {} weg:

```
const square = num => num * num;
```

2. Scope

2.1 Globale scope

- Variabelen buiten een functie of block zijn **globaal** en overal toegankelijk:

```
const color = "blue";  
function showColor() {  
  console.log(color); // blue  
}
```

2.2 Block scope

- Variabelen gedefinieerd met `let` of `const` binnen `{}` bestaan **alleen binnen dat blok**:

```
function logColor() {  
  let color = "pink";  
  console.log(color); // pink  
}  
console.log(color); // ReferenceError
```

Blokscope voorkomt **scope pollution** en maakt code beter beheersbaar.

2.3 Scope pollution

- Te veel globale variabelen of hergebruik van globale variabelen kan **bugs veroorzaken**:

```
let num = 50;  
function changeNum() {  
  num = 100; // globale variable overschreven  
}
```

2.4 Goede scoping

- Gebruik **lokale variabelen** tenzij ze echt globaal moeten zijn.
- Variabelen in een `if/else` of functie moeten **niet dezelfde naam** hebben als globale variabelen tenzij nodig.

```
const logLight = () => {  
  let light = "Moonlight";  
  if (true) {  
    let light = "Northern Lights"; // blokscope  
    console.log(light); // Northern Lights  
  }  
  console.log(light); // Moonlight  
}
```

3.2 Rock Paper Scissors

- Functies: `getUserChoice()`, `getComputerChoice()`, `determineWinner()`, `playGame()`.
- Gebruik van **default parameters**, **return**, en **helper functies**.

```
const getUserChoice = userInput => {  
  userInput = userInput.toLowerCase();  
  if (userInput === 'rock' || userInput === 'paper' || userInput === 'scissors') {
```

```

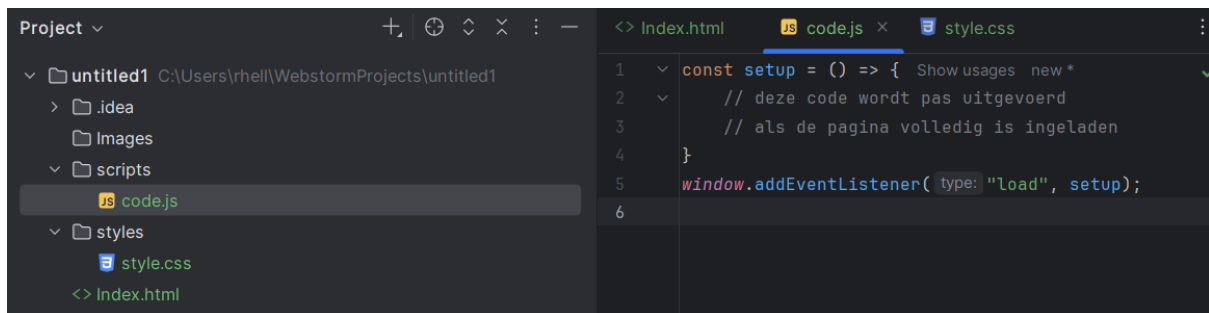
    return userInput;
  } else {
    console.log("Error: kies rock, paper of scissors");
  }
}

```

3.3 Sleep Debt Calculator

- Functies:
 - `getSleepHours(day)`: geeft slaapuren per dag
 - `getActualSleepHours()`: som van alle dagen
 - `getIdealSleepHours()`: ideale slaapuren
 - `calculateSleepDebt()`: berekent verschil en logt bericht
- Concepten: **functies, return, parameters, if/else.**

Opdracht 2



- Dit leeg project kun je als startpunt gebruiken van de opdrachten die je zult maken. Met andere woorden, voor elke opdracht kan je vanaf nu dit project kopiëren – wat je wat tijd bespaard.

Opdracht 3 Rekenmachine

JavaScript

```

// Setup functie: koppelt elke knop aan de juiste functie
const setup = () => {

```

```
document.getElementById("btnOptellen").addEventListener("click", optellen);
document.getElementById("btnAftrekken").addEventListener("click", aftrekken);
document.getElementById("btnVermenigvuldigen").addEventListener("click",
vermenigvuldigen);
document.getElementById("btnDelen").addEventListener("click", delen);
}
```

```
// Functie voor optellen
const optellen = () => {
  let g1 = parseFloat(document.getElementById("txtLinks").value);
  let g2 = parseFloat(document.getElementById("txtRechts").value);
  document.getElementById("txtOutput").innerHTML = `${g1} + ${g2} = ${g1 + g2}`;
}
```

```
// Functie voor aftrekken
const aftrekken = () => {
  let g1 = parseFloat(document.getElementById("txtLinks").value);
  let g2 = parseFloat(document.getElementById("txtRechts").value);
  document.getElementById("txtOutput").innerHTML = `${g1} - ${g2} = ${g1 - g2}`;
}
```

```
// Functie voor vermenigvuldigen
const vermenigvuldigen = () => {
  let g1 = parseFloat(document.getElementById("txtLinks").value);
  let g2 = parseFloat(document.getElementById("txtRechts").value);
  document.getElementById("txtOutput").innerHTML = `${g1} x ${g2} = ${g1 * g2}`;
}
```

```
// Functie voor delen
const delen = () => {
  let g1 = parseFloat(document.getElementById("txtLinks").value);
  let g2 = parseFloat(document.getElementById("txtRechts").value);

  if (g2 === 0) {
    document.getElementById("txtOutput").innerHTML = "Fout: delen door 0 is niet
mogelijk!";
    return;
  }

  document.getElementById("txtOutput").innerHTML = `${g1} : ${g2} = ${g1 / g2}`;
}
```

```
// Wacht tot de pagina geladen is voordat we de setup uitvoeren  
window.addEventListener('load', setup);
```

9 9 9 - 9 = 0